

Performance_Testing_Standard

Technical Standard

保险公司 技术开发部
内部文档 · 仅供授权人员查阅

Contents

- 1. Overview
 - 1.1 Purpose
 - 1.2 Scope
- 2. Test Types & Frequency
- 3. Performance Metrics
 - 3.1 Common Metrics
 - 3.2 Insurance Core System Targets
 - 3.3 Resource Targets
- 4. Scenario Design
 - 4.1 Scenario Template
 - 4.2 Mandatory Test Scenarios
- 5. Execution Process
 - 5.1 Preparation
 - 5.2 Execution
 - 5.3 Halt Conditions
- 6. Analysis & Report
 - 6.1 Report Template
- Summary
- Core Metrics Summary
- Resource Utilization
- Bottlenecks & Recommendations
- Capacity Estimate
- 6.2 Pass/Fail Criteria
- 7. Appendix
 - 7.1 Reference Documents
 - 7.2 Recommended Tools

Performance Testing Standard

> Reference: Alibaba Cloud PTS Best Practices, Tencent Full-Link Load Testing, Google Capacity Planning, JMeter Official Guide

1. Overview

1.1 Purpose

Standardize the performance testing process for insurance core systems, ensuring stable operation during peak seasons, promotional events, and other high-traffic scenarios.

1.2 Scope

Applies to all new system launches, major version iterations, and regular capacity assessment testing within the Technology Development Department.

2. Test Types & Frequency

Type	Trigger	Frequency
Regular	Major release, architecture change	Before each major release
Peak	Pre-peak season (e.g., New Year promotion)	1 month before peak
Capacity	Capacity planning, unexpected growth	Quarterly
Stability	Long-duration verification	Before major release
Chaos	Disaster recovery, chaos engineering	Semi-annually

3. Performance Metrics

3.1 Common Metrics

Metric	Definition	Target Reference
TPS / QPS	Transactions/Requests per second	See 3.2
RT (Response Time)	Time from request to response	P50 < 500ms, P99 < 2000ms
Error Rate	Failed request ratio	< 0.1% (core APIs)
Concurrent Users	Simultaneous online users	Estimated per scenario
Throughput (MB/s)	Network/disk throughput	< 60% of bandwidth

3.2 Insurance Core System Targets

Scenario	Peak QPS Target	P99 RT Target	Data Source
Online underwriting	500	< 1000ms	New Year campaign peak
Policy inquiry	2000	< 500ms	Agent app peak

Scenario	Peak QPS Target	P99 RT Target	Data Source
Claim reporting	100	< 2000ms	Post-disaster event
Payment callback	300	< 1000ms	Renewal payment day
Renewal batch	5000 (batch)	< 30min total	Renewal batch run

3.3 Resource Targets

Resource	Safe Level	Warning Level	Critical Level
CPU	< 50%	50%-70%	> 70%
Memory	< 60%	60%-80%	> 80%
Disk IO	< 40%	40%-70%	> 70%
Network	< 40%	40%-60%	> 60%
DB Connections	< 60%	60%-80%	> 80%
GC Frequency	---	FGC > 5/min	FGC pause > 1s

4. Scenario Design

4.1 Scenario Template

```
# Scenario: [Business Scenario Name]
# Requirement: [Requirement ID]
# Load Configuration
threads:
  initial: 10
  ramp_up: 30
  peak: 200
  duration: 300
# Endpoints
endpoints:
  - name: "Online Underwriting"
    url: "/v1/underwriting/evaluate"
    method: POST
    weight: 40
    data:
      source: "proposal_data.csv"
      strategy: "random"
  - name: "Policy Inquiry"
    url: "/v1/policy/{policy_no}"
    method: GET
    weight: 60
    data:
      source: "policy_no_list.csv"
      strategy: "sequential"
# Assertions
assertions:
  - response_time_p99_lt: 2000
  - error_rate_lt: 0.001
```

4.2 Mandatory Test Scenarios

No	Scenario	Description	Peak QPS
1	Submit > Underwriting > Issue	Core pipeline	500
2	Policy query (by number/ customer)	High-frequency read	2000
3	Claim report + document upload	File upload scenario	100
4	Payment callback processing	External system callback	300
5	Renewal batch payment	Batch processing	5000
6	Policy servicing + recalculation	Write operation	200
7	Login + auth verification	Mixed scenario baseline	800
8	Data report query	Complex query	50

5. Execution Process

5.1 Preparation

- ☐ Define test objectives (baseline vs limit vs stability)
- ☐ Set up test environment (proportional to production)
- ☐ Prepare test data (close to production scale)
- ☐ Deploy monitoring (APM / Prometheus / Grafana)
- ☐ Write test scripts
- ☐ Execute smoke test

5.2 Execution

1. **Baseline Test:** Single API, low concurrency (1-5 threads)
2. **Load Test:** Gradually increase concurrency, observe performance inflection point
3. **Peak Test:** Target peak load for 30 minutes
4. **Stability Test:** 70% peak load for 4-8 hours
5. **Stress Test:** Increase until system breaks or bottleneck found

5.3 Halt Conditions

- Error rate exceeds 5% and growing
- P99 RT exceeds 3x target
- CPU/Memory sustained above critical level
- DB replication lag exceeds 30 seconds
- Monitoring detects circuit breaker / degradation

6. Analysis & Report

6.1 Report Template

```
# Performance Test Report: [Project] v[Version]
## Summary
- Date: YYYY-MM-DD
- Environment: [Details]
- Tool: [Tool Name]
## Core Metrics Summary
| Scenario | Target QPS | Actual QPS | Target P99 | Actual P99 | Error Rate | Verdict |
| --- | --- | --- | --- | --- | --- | --- |
| Online UW | 500 | 520 | 1000ms | 850ms | 0.01% | Pass |
## Resource Utilization
| Service | CPU | Memory | GC | Notes |
| --- | --- | --- | --- | --- |
| policy-service | 45% | 3.2G/8G | FGC: 0 | Normal |
## Bottlenecks & Recommendations
1. [Bottleneck] > [Recommendation]
## Capacity Estimate
- Current capacity: [X] QPS
- 6-month projected need: [Y] QPS
- Recommended scaling: [Plan]
```

6.2 Pass/Fail Criteria

Verdict	Condition
Pass	All core scenarios meet QPS targets with P99 RT and error rate within thresholds
Conditional	Non-core scenarios below target with clear optimization plan
Fail	Core scenarios below target or performance bottleneck may cause production incidents

7. Appendix

7.1 Reference Documents

- JMeter Official Documentation
- *Full-Link Load Testing Guide*
- *Release Checklist Standard*
- Google SRE - Capacity Planning

7.2 Recommended Tools

Purpose	Tool	Alternative
Load Testing	JMeter / k6	Gatling / Locust
Full-Link Testing	Alibaba Cloud PTS	Tencent WeTest
APM	SkyWalking / Pinpoint	Datadog
Monitoring	Prometheus + Grafana	Zabbix
Tracing	Jaeger	Zipkin
Chaos Engineering	ChaosBlade	Litmus